



18. Naive Bayes Classifier

Learning Data with the Bayes' Theorem

Previous Section:

 [17. Hierarchical Clustering](#)

Contents:

[1. From Conditional Probability to The Bayes Theorem](#)

[2. Naive Bayes Classifier – Probabilistic Classification Model](#)

[3. Naive Bayes Classifier – The Workflow](#)

[4. Naive Bayes Classifier – The Zero-Frequency Problem](#)

[5. Naive Bayes Classifier with Python](#)

[Summary](#)

[References](#)

This section talks about Naive Bayes. Most machine learning models have probability under the hood, but no one really *commits* to probability the way Naive Bayes does.

Think about it like this. Most models *use* probability as a tool. Naive Bayes? It *lives* in probability. Everything it does—every prediction, every decision—is built directly on one simple but powerful idea: **the Bayes Theorem**. And what makes it even more interesting is this:

Naive Bayes doesn't try to be complicated. It doesn't build deep trees, complex boundaries, or layered representations. Instead, it asks a very direct question:

┆ *Given what I observe... what is the probability that this belongs to a certain class?*

And then it answers that question—purely through probability.

If you want to understand Naive Bayes properly, don't think of it as just another algorithm. Think of it as a shift in perspective.

- Instead of asking: "How do I draw the best boundary?"
- It asks: "How confident am I... that this data belongs to each possible class?"

And from that confidence, it makes a decision. That's it. Simple. Elegant. And surprisingly powerful.

1. From Conditional Probability to The Bayes Theorem

Let's start from something you already know. Conditional probability. At its core, it answers a very simple question:

┆ *What is the probability of A... given that B has already happened?*

And mathematically, we define it as:

$$P(A \mid B) = \frac{P(A \cap B)}{P(B)}$$

Now here's where things start to get interesting. If we take this definition, and rearrange the terms of the joint probability, we arrive at one of the most important equations in probability theory → The Bayes Theorem.

$$P(A | B) = \frac{P(B|A)P(A)}{P(B)}$$

Now don't rush past this formula. Every single term here has a meaning—and in Machine Learning, those meanings become *very* practical.

In the context of classification:

- **Posterior Probability** $P(A | B)$: This is what we ultimately care about. The probability that the data belongs to class A , *given what we observed*.
- **Likelihood** $P(B | A)$: How likely is the observed data if we assume it belongs to class A ?
- **Prior Probability** $P(A)$: How common is class A before seeing any data?
- **Evidence** $P(B)$: The overall probability of observing the data, across all classes. This term ensures everything stays properly normalized.



- The complement of $P(A)$ or $P(B)$ is denoted as $P(A^c)$ or $P(B^c)$.
- When conditioning on B , the world is already restricted to B . So the complement of $P(A | B)$ is: $P(A^c | B) \rightarrow$ **Not** $P(A | B^c)$ —because that would change the condition itself.

Now here's an important simplification. When we're doing classification, we don't actually need the exact value of $P(B)$. Why? Because it is the same for *all* classes.

So when comparing probabilities across classes, it acts like a constant—and we can safely ignore it. Which means, in practice, Naive Bayes is really comparing:

$$P(A | B) \propto P(B | A) \cdot P(A)$$

The Law of Total Probability: But let's not ignore $P(B)$ completely—because understanding it gives you a deeper picture.

The evidence term can be computed using the **Law of Total Probability**:

$$P(B) = \sum_{i=1}^n P(A_i)P(B | A_i)$$

Here, $A_1, A_2, \dots, A_n$ represent all possible classes.

What this tells us is something very intuitive:

▮ The probability of observing the data B is just the sum of all the ways it could happen under each class.

Each class contributes its prior probability, multiplied by how likely it explains the data

So in a way, every class is "competing" to explain the observation.

Naïve Independence Assumption: Now we move to the part where Naive Bayes becomes "*naive*". In real-world problems, the observed data B is not just a single value—it's a collection of features: $B = (x_1, x_2, \dots, x_m)$

So ideally, we would compute:

$$P(B | A) = P(x_1, x_2, \dots, x_m | A)$$

But here's the problem: This joint probability becomes incredibly complex as the number of features grows.

So Naive Bayes makes a bold assumption: *All features are conditionally independent given the class.*

And under this assumption, everything simplifies beautifully:

$$P(B | A) = \prod_{i=1}^m P(x_i | A)$$

Is this assumption realistic? Not really. In most real-world data, features are *not* independent. But here's the surprising part:

Even with this “naive” assumption, the model still works incredibly well.

Because it drastically reduces complexity. Instead of estimating one massive joint probability, we only need to estimate individual probabilities for each feature.

And that's what makes Naive Bayes fast, scalable, extremely effective for high-dimensional data (like text classification)

So when you look at Naive Bayes, don't just see a formula. You see the idea behind it: Take a complex probabilistic problem → Make a simplifying assumption → And turn something intractable into something practical.

2. Naive Bayes Classifier – Probabilistic Classification Model

Now that we understand the theory, let's bring it into something practical.

The **Naïve Bayes Classifier** is a probabilistic classification model built directly on Bayes' Theorem. And unlike many models that *hide* probability inside their mechanics, Naïve Bayes puts probability at the very center of everything it does.

It is especially popular in tasks like:

- Spam detection
- Sentiment analysis
- Document classification

And when the features are categorical (like words or symbols), we often call it: **Categorical Naïve Bayes**

At its core, the model does something very simple: It estimates the probability that a data instance belongs to each class, and then chooses the class with the highest probability.

Mathematically, under the independence assumption, we compute:

$$P(\text{Class} | \text{Features}) \propto P(\text{Class}) \prod_i P(\text{Feature}_i | \text{Class})$$

And this is where the name “naïve” comes from. It assumes that all features are **conditionally independent given the class**.

Is that realistic? Not at all. For some problems, the input features clearly depend on each other. But this assumption simplifies the problem so much, that the model becomes incredibly efficient—and still works surprisingly well.

Example: Spam Email Classification - Let's make this concrete.

Imagine we have a dataset of **50 emails**: 48 are **non-spam** and 2 are **spam**

So the prior probability is:

$$P(\text{NonSpam}) = \frac{48}{50}$$

Now suppose we look at how often certain words appear in **non-spam emails**:

Word	Occurrences
Dear	8

Word	Occurrences
My Friend	5
Hello	3
Good Day	1

Let's compute one example step by step. The joint probability that an email is non-spam **and** starts with "Dear" is:

$$P(\text{Dear} \cap \text{NonSpam}) = \frac{8}{50}$$

Using conditional probability:

$$P(\text{Dear} \mid \text{NonSpam}) = \frac{P(\text{Dear} \cap \text{NonSpam})}{P(\text{NonSpam})} = \frac{8/50}{48/50} = \frac{8}{48}$$

Similarly, we can compute:

- $P(\text{My Friend} \mid \text{NonSpam}) = \frac{5}{48}$
- $P(\text{Hello} \mid \text{NonSpam}) = \frac{3}{48}$
- $P(\text{Good Day} \mid \text{NonSpam}) = \frac{1}{48}$

Putting Everything Together - Now imagine a new email arrives. Naïve Bayes will:

1. Take the **prior probability** of each class
2. Multiply it by the **likelihoods** of the observed words
3. Compare the results across all classes

And then simply choose: **The class with the highest posterior probability**

Under the independence assumption, the posterior probability of a class is proportional to:

$$P(\text{Class} \mid \text{Words}) \propto P(\text{Class}) \prod_i P(\text{Word}_i \mid \text{Class})$$

Why Naïve Bayes Works So Well

At this point, you might be thinking: **"This assumption is clearly wrong... so why does it work?"**. That's the interesting part. Even though the independence assumption is unrealistic, Naïve Bayes still performs remarkably well in many real-world problems.

Its strengths include:

- **Simplicity** → Easy to understand and implement
- **Speed** → Fast training and prediction
- **Robustness** → Handles irrelevant features surprisingly well
- **Scalability** → Works great in high-dimensional data (like text)

So in a way, Naïve Bayes teaches an important lesson: You don't always need a perfect model, you need a model that is *simple enough to work, and powerful enough to generalize*.

And this makes Naïve Bayes a perfect bridge: From **pure probability theory** → to **practical machine learning systems**.

3. Naive Bayes Classifier – The Workflow

Now that we've understood that the Naïve Bayes Classifier is fundamentally a **probabilistic model built on Bayes' Theorem**, we can finally look at how it actually *operates* in practice.

At its core, the workflow is surprisingly simple—but also very powerful.

- **Step 1: Learn from the training data:** Instead of trying to directly “decide” the class, Naïve Bayes first **learns probabilities** from the data:

- The **prior probability**: $P(Y) \rightarrow$ How likely is each class overall? (e.g., how often do we see $Play = Yes$ vs $Play = No$)
- The **conditional probability**: $P(X | Y) \rightarrow$ How likely are the features given a class? (e.g., how likely is *Sunny* when $Play = Yes$)

→ So rather than memorizing examples, the model builds a **probabilistic understanding of the data**.

- **Step 2: Represent a new data point:** When a new, unseen instance comes in:

- We denote it as: $X_{\text{new}} = (x_1, x_2, \dots, x_n)$
- Each x_i is a feature value (e.g., *Outlook = Sunny*, *Humidity = High*, etc.)

→ The question now becomes: “Given these features... which class is more likely?”

- **Step 3: Apply the Naïve assumption:** Here’s where the “Naïve” part comes in.

- Instead of trying to model complex relationships between features (which can be very hard), we assume: **All features are conditionally independent given the class.**
- This simplifies the joint probability dramatically:

$$P(X | Y) = \prod_{i=1}^n P(x_i | Y)$$

This assumption may not always be perfectly true in reality. But it makes the model **computationally efficient** and often surprisingly effective.

- **Step 4: Compute the posterior probability** - Now we combine everything using Bayes’ idea:

$$P(Y | X_{\text{new}}) \propto P(Y) \prod_{i=1}^n P(x_i | Y)$$

Notice something important here: We don’t actually need the denominator $P(X)$. Because it’s the same for all classes. So we only compute the **proportional score** for each class.

- **Step 5: Make the final decision** - After computing this value for all possible classes:

- Choose the class with the **highest posterior probability**

$$\hat{Y} = \arg \max_Y P(Y) \prod_{i=1}^n P(x_i | Y)$$

→ This is the final prediction.

If we step back and look at this from my point of view, the philosophy becomes very clear:

Naïve Bayes doesn’t try to understand the world in a complex way. It breaks everything down into simple, independent pieces, and lets probability do the work.

- It **observes frequencies** → builds beliefs
- It **assumes simplicity** → avoids complexity
- It **reasons under uncertainty** → makes the best possible guess

And that’s why, despite its “naïve” assumption, it works so well—especially in domains like text classification, where simplicity and probability often beat overcomplicated models.

The workflow of Naïve Bayes can be seen as a clean pipeline:

Learn probabilities → Apply independence → Compute posterior → Choose the most likely class

Or in one sentence: “Given what I’ve seen before... which class makes this new data most probable?”

Now we will demonstrate the full and detail Naive Bayes workflow with this dataset:

Subject	Body Length	Email Type	Contains Links	Target (Status)
Marketing	Statement	Informal	NO	Spam
Marketing	Statement	Informal	YES	Spam
Operational	Statement	Informal	NO	Not Spam
Transactional	Paragraph	Informal	NO	Not Spam
Transactional	Essay	Formal	NO	Not Spam
Transactional	Essay	Formal	YES	Spam
Operational	Essay	Formal	YES	Not Spam
Marketing	Paragraph	Informal	NO	Spam
Marketing	Essay	Formal	NO	Not Spam
Transactional	Paragraph	Formal	NO	Not Spam
Marketing	Paragraph	Formal	YES	Not Spam
Operational	Paragraph	Informal	YES	Not Spam
Operational	Statement	Formal	NO	Not Spam
Transactional	Paragraph	Informal	YES	Spam

Number of Samples = 14. The target variable is **Status**, with possible values **Spam** or **Not Spam**.

- **Step 1: Estimate Prior Probabilities $P(Y)$**

- From the dataset we have 5 emails labeled as Spam, and 9 emails labeled as Not Spam. Therefore:

$$P(Y : Spam) = 5/14; P(Y : NotSpam) = 9/14$$

- **Step 2: Estimate Conditional Probabilities for each labeled feature $P(X|Y)$**

- For Feature = Subject (X_1)

Subject = *Marketing* (3 Spam, 2 Not Spam)

$$P(X_1 : Marketing \wedge Y : Spam) = 3/14$$

$$\rightarrow P(X_1 : Marketing|Y : Spam) = \frac{P(X_1:Marketing \wedge Y:Spam)}{P(Y:Spam)} = \frac{3/14}{5/14} = 3/5$$

$$P(X_1 : Marketing \wedge Y : NotSpam) = 2/14$$

$$\rightarrow P(X_1 : Marketing|Y : NotSpam) = \frac{P(X_1:Marketing \wedge Y:NotSpam)}{P(Y:NotSpam)} = \frac{2/14}{9/14} = 2/9$$

Subject = *Operational* (0 Spam, 4 Not Spam)

$$P(X_1 : Operational \wedge Y : Spam) = 0/14$$

$$\rightarrow P(X_1 : Operational|Y : Spam) = 0$$

$$P(X_1 : Operational \wedge Y : NotSpam) = 4/14$$

$$\rightarrow P(X_1 : Operational|Y : NotSpam) = \frac{P(X_1:Operational \wedge Y:NotSpam)}{P(Y:NotSpam)} = \frac{4/14}{9/14} = 4/9$$

Subject = *Transactional* (2 Spam, 3 Not Spam)

$$P(X_1 : Transactional \wedge Y : Spam) = 2/14$$

$$\rightarrow P(X_1 : Transactional|Y : Spam) = \frac{P(X_1:Transactional \wedge Y:Spam)}{P(Y:Spam)} = \frac{2/14}{5/14} = 2/5$$

$$P(X_1 : Transactional \wedge Y : NotSpam) = 3/14$$

$$\rightarrow P(X_1 : Transactional|Y : NotSpam) = \frac{P(X_1:Transactional \wedge Y:NotSpam)}{P(Y:NotSpam)} = \frac{3/14}{9/14} = 3/9$$

- For Feature = Body Length (X_2)

Body Length = *Statement* (2 Spam, 2 Not Spam)

$$P(X_2 : \text{Statement} \wedge Y : \text{Spam}) = 2/14$$

$$\rightarrow P(X_2 : \text{Statement} | Y : \text{Spam}) = \frac{P(X_2 : \text{Statement} \wedge Y : \text{Spam})}{P(Y : \text{Spam})} = \frac{2/14}{5/14} = 2/5$$

$$P(X_2 : \text{Statement} \wedge Y : \text{NotSpam}) = 2/14$$

$$\rightarrow P(X_2 : \text{Statement} | Y : \text{NotSpam}) = \frac{P(X_2 : \text{Statement} \wedge Y : \text{NotSpam})}{P(Y : \text{NotSpam})} = \frac{2/14}{9/14} = 2/9$$

Body Length = *Paragraph* (2 Spam, 4 Not Spam)

$$P(X_2 : \text{Paragraph} \wedge Y : \text{Spam}) = 2/14$$

$$\rightarrow P(X_2 : \text{Paragraph} | Y : \text{Spam}) = \frac{P(X_2 : \text{Paragraph} \wedge Y : \text{Spam})}{P(Y : \text{Spam})} = \frac{2/14}{5/14} = 2/5$$

$$P(X_2 : \text{Paragraph} \wedge Y : \text{NotSpam}) = 4/14$$

$$\rightarrow P(X_2 : \text{Paragraph} | Y : \text{NotSpam}) = \frac{P(X_2 : \text{Paragraph} \wedge Y : \text{NotSpam})}{P(Y : \text{NotSpam})} = \frac{4/14}{9/14} = 4/9$$

Body Length = *Essay* (1 Spam, 3 Not Spam)

$$P(X_2 : \text{Essay} \wedge Y : \text{Spam}) = 1/14$$

$$\rightarrow P(X_2 : \text{Essay} | Y : \text{Spam}) = \frac{P(X_2 : \text{Essay} \wedge Y : \text{Spam})}{P(Y : \text{Spam})} = \frac{1/14}{5/14} = 1/5$$

$$P(X_2 : \text{Essay} \wedge Y : \text{NotSpam}) = 4/14$$

$$\rightarrow P(X_2 : \text{Essay} | Y : \text{NotSpam}) = \frac{P(X_2 : \text{Essay} \wedge Y : \text{NotSpam})}{P(Y : \text{NotSpam})} = \frac{3/14}{9/14} = 3/9$$

- For Feature = Email Type (X_3)

Email Type = *Informal* (4 Spam, 3 Not Spam)

$$P(X_3 : \text{Informal} \wedge Y : \text{Spam}) = 4/14$$

$$\rightarrow P(X_3 : \text{Informal} | Y : \text{Spam}) = \frac{P(X_3 : \text{Informal} \wedge Y : \text{Spam})}{P(Y : \text{Spam})} = \frac{4/14}{5/14} = 4/5$$

$$P(X_3 : \text{Informal} \wedge Y : \text{NotSpam}) = 3/14$$

$$\rightarrow P(X_3 : \text{Informal} | Y : \text{NotSpam}) = \frac{P(X_3 : \text{Informal} \wedge Y : \text{NotSpam})}{P(Y : \text{NotSpam})} = \frac{3/14}{9/14} = 3/9$$

Email Type = *Formal* (1 Spam, 6 Not Spam)

$$P(X_3 : \text{Formal} \wedge Y : \text{Spam}) = 1/14$$

$$\rightarrow P(X_3 : \text{Formal} | Y : \text{Spam}) = \frac{P(X_3 : \text{Formal} \wedge Y : \text{Spam})}{P(Y : \text{Spam})} = \frac{1/14}{5/14} = 1/5$$

$$P(X_3 : \text{Formal} \wedge Y : \text{NotSpam}) = 6/14$$

$$\rightarrow P(X_3 : \text{Formal} | Y : \text{NotSpam}) = \frac{P(X_3 : \text{Formal} \wedge Y : \text{NotSpam})}{P(Y : \text{NotSpam})} = \frac{6/14}{9/14} = 6/9$$

- For Feature = Contain Links (X_4)

Contain Links = *Yes* (3 Spam, 3 Not Spam)

$$P(X_4 : \text{Yes} \wedge Y : \text{Spam}) = 3/14$$

$$\rightarrow P(X_4 : \text{Yes} | Y : \text{Spam}) = \frac{P(X_4 : \text{Yes} \wedge Y : \text{Spam})}{P(Y : \text{Spam})} = \frac{3/14}{5/14} = 3/5$$

$$P(X_4 : \text{Yes} \wedge Y : \text{NotSpam}) = 3/14$$

$$\rightarrow P(X_4 : \text{Yes} | Y : \text{NotSpam}) = \frac{P(X_4 : \text{Yes} \wedge Y : \text{NotSpam})}{P(Y : \text{NotSpam})} = \frac{3/14}{9/14} = 3/9$$

Contain Links = *No* (2 Spam, 6 Not Spam)

$$P(X_4 : \text{No} \wedge Y : \text{Spam}) = 2/14$$

$$\rightarrow P(X_4 : \text{No} | Y : \text{Spam}) = \frac{P(X_4 : \text{No} \wedge Y : \text{Spam})}{P(Y : \text{Spam})} = \frac{2/14}{5/14} = 2/5$$

$$P(X_4 : \text{No} \wedge Y : \text{NotSpam}) = 6/14$$

$$\rightarrow P(X_4 : \text{No} | Y : \text{NotSpam}) = \frac{P(X_4 : \text{No} \wedge Y : \text{NotSpam})}{P(Y : \text{NotSpam})} = \frac{6/14}{9/14} = 6/9$$



You can simplify the calculation process involving a feature with only two classes by using the complementary rule:

The positive class is X and the negative class is X^c

→ $P(X^c|Y) = 1 - P(X|Y)$. But it is your choice, not mine

Summary Table:

Feature X_i	Category	$P(X_i : Category Spam)$	$P(X_i : Category NotSpam)$
Subject	Marketing	3/5	2/9
	Operational	0	4/9
	Transactional	2/5	3/9
Body Link	Statement	2/5	2/9
	Paragraph	2/5	4/9
	Essay	1/5	3/9
Email Type	Informal	4/5	3/9
	Formal	1/5	6/9
Contain Links	Yes	3/5	3/9
	No	2/5	6/9

But here is a catch, and it's a serious one. At this point, it *looks* like we're ready to move on to prediction. We have: $P(Y)$ and $P(x_i | Y)$ → So naturally, the next step is:

$$P(Y | X) \propto P(Y) \prod_{i=1}^n P(x_i | Y)$$

Everything seems clean. But here's the problem → **The multiplication trap**

Naïve Bayes doesn't **add** probabilities. It **multiplies** them. And that tiny detail changes everything. Because multiplication has a very unforgiving rule:

| If any term is 0 → the entire result becomes 0

Remember the core assumption: *All features are conditionally independent given the class.*

So instead of computing a complex joint probability, we simplify:

$$P(x_1, x_2, \dots, x_n | Y) = P(x_1 | Y) \cdot P(x_2 | Y) \cdots P(x_n | Y)$$

This is what makes Naïve Bayes fast and scalable. But it also creates a dangerous side effect:

$$\text{If } P(x_i | Y) = 0 \Rightarrow P(Y | X) = 0$$

No matter how strong the other features are... That class is **completely eliminated**.

→ This is known in the Naive Bayes language as **The Zero-Frequency Problem**

Now look back at our table: $P(\text{Subject} = \text{Operational} | \text{Spam}) = 0$

Does this mean: *"Operational emails can never be spam"*? Of course not. It just means: *"We never saw it in this dataset."* → And that's the key distinction.

The model is not being logical here, it's being literal. It doesn't question the data. It doesn't consider uncertainty. It simply says: *"I have never seen this happen, so it must be impossible"*.

This problem appears frequently when the dataset is **small**, the feature space is **large**, or some values are **rare**. And when it happens, Naïve Bayes becomes **overconfident, fragile, and unrealistic**. One missing observation and an entire class gets erased.

"Naïve Bayes simplifies the world to make learning possible... But sometimes, that simplicity turns absence into impossibility".

And that's a mistake we'll need to fix next.

4. Naive Bayes Classifier – The Zero-Frequency Problem

The **Zero-Frequency Problem** is not just a minor inconvenience—it is a **critical issue** that must be addressed *before* making any prediction with Naïve Bayes. From the previous section, we saw the danger clearly:

| If any $P(x_i | Y) = 0 \rightarrow$ the entire posterior becomes 0

That means the model can completely discard a class, simply because it has **never seen a particular feature value before** $\rightarrow$ Our objective is to change from **"never seen"** to **"possibly seen"**.

So how do we fix this? We don't try to collect infinite data. We don't redesign the model. Instead, we make a small—but powerful—adjustment:

| *"We assume that every possible feature value has been seen... at least once".*

This idea is implemented through **Laplace Smoothing** (also known as **Add-One Smoothing**).

Where does this idea come from? Additive Smoothing (a popular technique in Large Language Models).

Conceptually, you can think of it as a very simple form of **additive correction**: We slightly adjust the counts so that nothing ever remains at zero.

In this context: We use $\alpha = 1 \rightarrow$ Which leads to what we call **Laplace smoothing**

Instead of trusting the raw counts directly, we *pretend*: Every feature value appears at least once in every class. This prevents the model from ever saying: "This is impossible."

The Laplace-smoothed formula

For categorical features, the adjusted probability becomes:

$$P(\text{Feature} = x \mid \text{Class} = c) = \frac{\text{Count}(x, c) + 1}{\text{Count}(c) + k}$$

Where:

- $\text{Count}(x, c) \rightarrow$ number of times feature value x appears in class c
- $\text{Count}(c) \rightarrow$ total number of samples in class c
- $k \rightarrow$ number of unique possible values for that feature

What actually changes? Two things happen simultaneously:

1. **Numerator increases by 1** $\rightarrow$ No count is ever zero anymore
2. **Denominator increases by k** $\rightarrow$ Keeps the probabilities properly normalized

So we are not just "adding 1" blindly— we are carefully **redistributing probability mass**.

After smoothing:

- All probabilities become **strictly positive**
- The model can now handle **unseen feature values**
- Predictions become **more stable**, especially with small datasets

But there is a trade-off. Laplace smoothing is not "free." $\rightarrow$ It introduces a **small bias**:

| *We are slightly distorting the true frequencies*

But in return, we gain something far more important: Robustness; generalization; and real-world reliability

What happens as data grows? This is the elegant part. As the dataset becomes larger:

- The "+1" becomes insignificant

- The smoothed probabilities gradually approach the true values

Smoothed estimate → Empirical probability

So the method **self-corrects over time**.

Laplace smoothing doesn't change what the model knows. It changes how confident the model is about what it doesn't know. Instead of saying: "I've never seen this → impossible"

The model now says: *"I haven't seen this... but I'll keep a small probability."* And that single shift is what makes Naïve Bayes usable in the real world.

Back to the problem: Now that we have the formula, let's return to what we are actually trying to fix.

The goal is not just to "remove zeros" → It is to **adjust the entire probability distribution**: $P(X | Y)$ and $P(X^c | Y)$, so that we avoid extreme situations like: $P(X | Y) = 0$ and $P(X^c | Y) = 1$

These values look confident, but they are often **misleading**. They don't reflect reality. They reflect **missing data**.

Laplace smoothing is not just a fix. It is a **redistribution of probability mass**. We take a tiny amount from "certain" events and give it to "unseen" events

So instead of: **Impossible vs Certain** → We move to: **Unlikely vs Likely**

Unseen does not mean impossible. It means... we haven't seen it yet.

So we treat unseen events as: **Rare, but still possible**

The important catch (and this is critical)

You might think: *"Okay, just fix the zero... and leave everything else as it is."* → That would be a **huge** mistake.

Smoothing is NOT selective: If one probability needs smoothing, **then ALL probabilities must be smoothed**. Not just the zero ones.

Why? Because probabilities must remain **balanced, comparable, and summing to 1**

If you only adjust one value, you break the distribution, you distort relationships, you introduce bias. Basically, smoothing is saying:

If I am updated... you are updated too.

Every feature value is part of the same system. You cannot change one without affecting the others.

Laplace smoothing is not about fixing a mistake. It's about enforcing a principle:

Every possibility deserves a non-zero voice.

And that voice must be given **fairly to all**—not just to the ones that were missing.

- **Applying Laplace Smoothing: For every feature x_i → Class c_i**
 - Determine k , which basically means how many unique labels c are in each feature - also very important for smoothing).
 - Find the $Count(x_i, c_i)$ and $Count(c)$
 - Apply the smoothed formula

We now continue the smoothing process for **all features**. Remember the rule: *If one value is updated → all values must be updated.*

Feature X_1 : Subject ($k = 3$)

$Count(Spam) = 5, Count(NotSpam) = 9$

$Counts \rightarrow Feature : (Spam \ counts, NotSpam \ counts):$

- Marketing: (3, 2)
- Operational: (0, 4)
- Transactional: (2, 3)

Smoothed:

- $P(\text{Marketing} \mid \text{Spam}) = \frac{3+1}{5+3} = 4/8$
- $P(\text{Operational} \mid \text{Spam}) = \frac{0+1}{5+3} = 1/8$
- $P(\text{Transactional} \mid \text{Spam}) = \frac{2+1}{5+3} = 3/8$
- $P(\text{Marketing} \mid \text{NotSpam}) = \frac{2+1}{9+3} = 3/12$
- $P(\text{Operational} \mid \text{NotSpam}) = \frac{4+1}{9+3} = 5/12$
- $P(\text{Transactional} \mid \text{NotSpam}) = \frac{3+1}{9+3} = 4/12$

Feature X_2 : Body Length ($k = 3$)

- Statement: (2, 2)
- Paragraph: (2, 4)
- Essay: (1, 3)

Smoothed:

- $P(\text{Statement} \mid \text{Spam}) = \frac{2+1}{5+3} = 3/8$
- $P(\text{Paragraph} \mid \text{Spam}) = \frac{2+1}{5+3} = 3/8$
- $P(\text{Essay} \mid \text{Spam}) = \frac{1+1}{5+3} = 2/8$
- $P(\text{Statement} \mid \text{NotSpam}) = \frac{2+1}{9+3} = 3/12$
- $P(\text{Paragraph} \mid \text{NotSpam}) = \frac{4+1}{9+3} = 5/12$
- $P(\text{Essay} \mid \text{NotSpam}) = \frac{3+1}{9+3} = 4/12$

Feature X_3 : Email Type ($k = 2$)

- Informal: (4, 3)
- Formal: (1, 6)

Smoothed:

- $P(\text{Informal} \mid \text{Spam}) = \frac{4+1}{5+2} = 5/7$
- $P(\text{Formal} \mid \text{Spam}) = \frac{1+1}{5+2} = 2/7$
- $P(\text{Informal} \mid \text{NotSpam}) = \frac{3+1}{9+2} = 4/11$
- $P(\text{Formal} \mid \text{NotSpam}) = \frac{6+1}{9+2} = 7/11$

Feature X_4 : Contains Links ($k = 2$)

- Yes: (3, 3)
- No: (2, 6)

Smoothed:

- $P(\text{Yes} \mid \text{Spam}) = \frac{3+1}{5+2} = 4/7$
- $P(\text{No} \mid \text{Spam}) = \frac{2+1}{5+2} = 3/7$
- $P(\text{Yes} \mid \text{NotSpam}) = \frac{3+1}{9+2} = 4/11$
- $P(\text{No} \mid \text{NotSpam}) = \frac{6+1}{9+2} = 7/11$

We will return to update the summary table:

Feature X_i	Category	$Count(X_i, Spam)$	$Count(X_i, NotSpam)$	Smoothed $P(X_i : Category Spam)$	Smoothed $P(X_i : Category NotSpam)$
Subject	Marketing	3	2	4/8	3/12
$k = 3$	Operational	0	4	1/8	5/12
	Transactional	2	3	3/8	4/12
Body Link	Statement	2	2	3/8	3/12
$k = 3$	Paragraph	2	4	3/8	5/12
	Essay	1	3	2/8	4/12
Email Type	Informal	4	3	5/7	4/11
$k = 2$	Formal	1	6	2/7	7/11
Contain Links	Yes	3	3	4/7	4/11
$k = 2$	No	2	6	3/7	7/11



One tip to help you find the $count(x_i, c_i)$ quickly → Look at the **numerator** of the $P(X_i|Y)$ for every feature X_i

Notice what just happened: No probability is 0 anymore, and no probability is 1 anymore. Every value is now **possible**. And more importantly: “*The relationships are preserved... but the rigidity is gone*”.

This is the real purpose of smoothing. Not to change the model— but to make it **less naive about uncertainty**.

Making Prediction (The moment of truth)

Subject	Body Length	Email Type	Contains Links	Target (Status)
Marketing	Essay	Formal	Yes	?

- Step 1: Compute $P(X | Y)$

$$P(X | Y : Spam) = P(X_1 : Marketing|Y : Spam) \times P(X_2 : Essay|Y : Spam) \times P(X_3 : Formal|Y : Spam) \times P(X_4 : Yes|Y : Spam)$$

$$= (4/8) \times (2/8) \times (2/7) \times (4/7) = 1/49 = 0.0204$$

$$P(X | Y : NotSpam) = P(X_1 : Marketing|Y : NotSpam) \times P(X_2 : Essay|Y : NotSpam) \times P(X_3 : Formal|Y : NotSpam) \times P(X_4 : Yes|Y : NotSpam)$$

$$= (3/12) \times (4/12) \times (7/11) \times (4/11) = 7/363 = 0.01928$$

- Step 2: Compute $P(Y | X_{new})$

$$P(Y : Spam|X_{new}) = P(Y : Spam) \times P(X | Y : Spam)$$

$$= (5/14) \times (1/49) = (5/686) = 0.00729$$

$$P(Y : NotSpam|X_{new}) = P(Y : NotSpam) \times P(X | Y : NotSpam)$$

$$= (9/14) \times (7/363) = (3/242) = 0.012397$$

- Step 3: Final prediction

Since: $P(Y : Spam|X_{new}) < P(Y : NotSpam|X_{new}) = 0.00729 < 0.012397 \rightarrow$ The email is classified as

NotSpam

5. Naive Bayes Classifier with Python

Here's how you would build a Naive Bayes Classifier model in Python using the Scikit-Learn library. Using a toy Email Classification Dataset.

```
import pandas as pd

# Load the dataset
data = {"Subject": ["Marketing", "Marketing", "Operational", "Transactional", "Transactional",
                  "Transactional", "Operational", "Marketing", "Marketing", "Transactional",
                  "Marketing", "Operational", "Operational", "Transactional"],
        "Body Length": ["Statement", "Statement", "Statement", "Paragraph", "Essay",
                        "Essay", "Essay", "Paragraph", "Essay", "Paragraph",
                        "Paragraph", "Paragraph", "Statement", "Paragraph"],
        "Email Type": ["Informal", "Informal", "Informal", "Informal", "Formal",
                       "Formal", "Formal", "Informal", "Formal", "Formal",
                       "Formal", "Informal", "Formal", "Informal"],
        "Contains Links": ["NO", "YES", "NO", "NO", "NO",
                           "YES", "YES", "NO", "NO", "NO",
                           "YES", "YES", "NO", "YES"],
        "Target": ["Spam", "Spam", "Not Spam", "Not Spam", "Not Spam",
                   "Spam", "Not Spam", "Spam", "Not Spam", "Not Spam",
                   "Not Spam", "Not Spam", "Not Spam", "Spam"]}

df = pd.DataFrame(data)

# Label Encoding for demonstration only
for i in df.columns:
    unique_columns = df[i].unique()
    df[i] = df[i].map({unique_columns[value]: value for value in range(len(unique_columns))})
```

```
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import CategoricalNB
from sklearn.metrics import accuracy_score

X = df.drop("Target", axis=1)
y = df["Target"]

# Splitting data
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.4, random_state=42)
# Fit the model
model = CategoricalNB()
model.fit(X_train, y_train)
# Evaluate
y_pred = model.predict(X_test)
accuracy = accuracy_score(y_test, y_pred)
print("Accuracy Score:", accuracy)
```

Accuracy Score: 0.8333333333333334

```
# Calculate probabilities
y_proba = model.predict_proba(X_test)
# Display results
print("Probabilities for each test prediction:")
for i, (pred, prob) in enumerate(zip(y_pred, y_proba)):
    print(f"Sample {i}; Probability: {prob}")
    print(f"+ Predicted: {pred}\n+ Actual Label: {y_test.iloc[i]}\n")
```

```
Probabilities for each test prediction:
Sample 0; Probability: [0.14837245 0.85162755]
+ Predicted: 1
+ Actual Label: 1
Sample 1; Probability: [0.48164644 0.51835356]
+ Predicted: 1
+ Actual Label: 1
Sample 2; Probability: [0.67644521 0.32355479]
+ Predicted: 0
+ Actual Label: 0
Sample 3; Probability: [0.08013083 0.91986917]
+ Predicted: 1
+ Actual Label: 1
Sample 4; Probability: [0.10406159 0.89593841]
+ Predicted: 1
+ Actual Label: 0
Sample 5; Probability: [0.11556604 0.88443396]
+ Predicted: 1
+ Actual Label: 1
```

Summary

Naïve Bayes may look simple, almost *too* simple. But that simplicity is exactly why it is powerful.

It doesn't try to model everything. It focuses on what matters → probability, structure, and decision.

Despite its strong assumptions: It scales well. It works surprisingly effectively. It handles real-world problems like text classification with ease

And with techniques like smoothing, it becomes even more robust.

"Simple... but reliable. Naïve... but powerful".

That's why Naïve Bayes is not just simple, it's golden.

References

<https://www.geeksforgeeks.org/machine-learning/naive-bayes-classifiers/>

<https://www.datacamp.com/tutorial/naive-bayes-scikit-learn>

<https://www.ibm.com/think/topics/naive-bayes>

<https://www.youtube.com/watch?v=O2L2Uv9pdDA>

Next Section:

 [19. Gaussian Naive Bayes](#)